

Research Paper Presentation

Comparing 2 papers:

- *"A*Grid2D: Optimized Grid-Based A* Pathfinding for 2D Game Environments"* Written by: Randy S. Tolentino
- *"Identifying Artificial Intelligence Pathfinding Algorithms for Platformer Games"* Written by: Umar Affandi Sahrin Iskandar, Norizan Mat Diah and Marina Ismail

Presented by

Christopher Kulak

Zhire Pelayo

Table of Contents

1 Introduction



2 Background



3 Methodology



4 Data Analysis + Results



5 Discussion



6 Take Away



Introduction

- AI is one of the key components of video games. NPC's need to act in a goal oriented manner, avoiding obstacles while achieving their goal (finding the player, running away, etc)
- AI Pathfinding is a staple of video games, enemy ai pathfinding is in almost every game imaginable. Pathfinding is essentially attempting to find the shortest route between two points.



Background

- Our first papers objective is to review the existing pathfinding algorithms that have been used in video games.
- 2 types: Directed and Undirected pathfinding.
 - Undirected: There is no method to assess progress when picking an adjacent node to advance to. Algorithm may never find an answer to the problem determining the shortest path.
 - Essentially blindly going through a maze
- Undirected algorithms include
 - BFS (breadth first search)
 - DFS (depth first search)



Background Continued

- Directed Pathfinding-all directed pathfinding algorithms have one thing in common, they don't blindly go through a maze, they can assess their progress from all adjacent nodes.
- These algorithms will nearly always find a solution.
 - Uniform cost search-always choose lowest cost node
 - Heuristic search-estimate cost of the next node.
- These have the problem of being unable to handle dynamic worlds, and can't produce realistic movements from the NPCs



Pathfinding, the common algorithms

The most common game algorithms: Dijkstra's algorithm, A*, DFS, BFS, Best First Search and D*

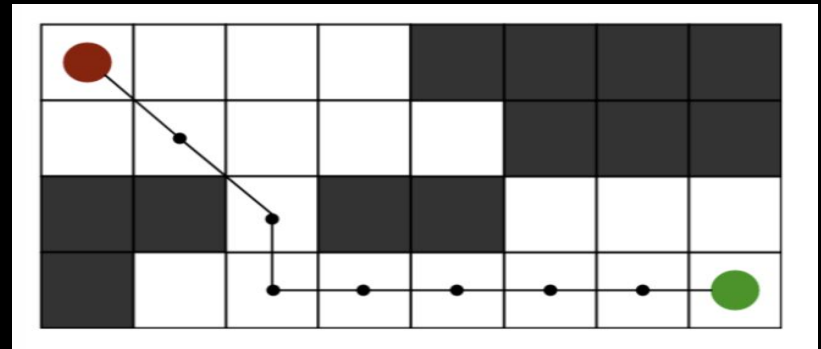
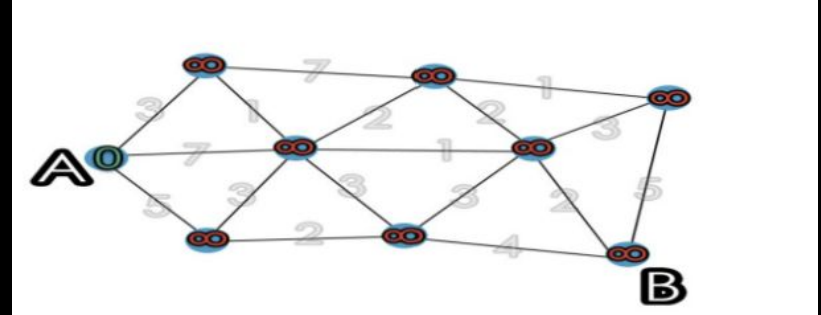
A*-informed search algorithm combining features of Dijkstra and Best First Search. Formulated in terms of weighted graphs in which it picks the nodes at each step.

D*-Dynamic variant of A* which is optimized for partially obscured environments.

DFS-traversing or searching a tree or graph data structures (randomly select starting node) uses a bool to keep track of visited nodes

BFS-traversing all nearby nodes from the root node

Best first search-a node is selected for expansion based on a evaluation function $f(n)$. Doesn't guarantee shortest path, but significantly faster execution speed.



Algorithm	Advantage	Disadvantage
A*	- Will always find the solution and shortest path - Will only focus on the target node - Expands the fewest number of nodes to find the optimal path	- Complex to implement - Not optimal when there exists several target - Consumes a lot of memory - Has complexity problems as speed of execution highly depends on accuracy of Heuristics.

Depth First Search	- Simple to implement - Small memory needed to use it - If depth-first search finds solution without exploring much in a path then the time and space it takes will be very less.	- May fail to find a solution and get stuck in an infinite long branch - Not guaranteed to find optimal solution - Take the longest time to find the solution on bigger maps among the found algorithms
Best First Search	- More Efficient than BFS or DFS - Has the benefits of both DFS and BFS - Better time complexity than BFS	May cover more distance than is intended

Dijkstra	- Will always find the shortest path - Picks edges with the smallest cost and is useful when there are multiple target nodes - Dijkstra will always fit while A* will require an appropriate heuristics function to be available.	- Blind search may consume time - It cannot handle negative edges - May fail if negative edges are involved
Breadth-First Search	- Breadth first search will never get trapped exploring the useless path forever. - BFS will definitely find a solution if one exists. - Is able to determine the solution that requires less steps	- Has major memory issues - Solution finding will consume a lot of time if its farther away - Is a lot more complex to implement

Details and drawbacks

- A* Pathfinding
 - Search algorithm that combines features of Dijkstra and Best First Search
 - $F = G + H$ where
 - F is the sum of G and H
 - G is the cost of movement in order to move from starting to end location
 - H is the estimate cost of the cheapest path from starting to end location

Methodology

- A*Grid2D is an improvement over A*: "Integrating discrete path computation, specialized heuristics, and adaptive obstacle tracking, ensuring that NPCs react realistically to dynamic environments." (Tolentino, 2025).
- The study was applied to three different genre of games
 - Lost Cat - platformer
 - Antivirus - top-down shooter
 - Dungeon Escape - dungeon crawler
- QA testing was conducted to identify bottlenecks and bugs affecting efficiency and responsiveness.

Data Analysis + Results

Components	A* Algorithm	Dijkstra's Algorithm	BFS Algorithm
Length	38	38	38
Time	0.3500ms	0.3000ms	0.8000ms
Computed Blocks	323 blocks	738 blocks	738 blocks

Conclusion

Algorithm	Advantage	Disadvantage
A*	• Will always find the solution and shortest path • Will only focus on the target node • Expands the fewest number of nodes to find the optimal path	• Complex to implement • Not optimal when there exists several target • Consumes a lot of memory • Has complexity problems as speed of execution highly depends on accuracy of Heuristics.

Dijkstra	• Will always find the shortest path • Picks edges with the smallest cost and is useful when there are multiple target nodes • Dijkstra will always fit while A* will require an appropriate heuristics function to be available. •	• Blind search may consume time • It cannot handle negative edges • May fail if negative edges are involved
----------	--	---

Best First Search	• More Efficient than BFS or DFS • Has the benefits of both DFS and BFS • Better time complexity than BFS	• May cover more distance than is intended
-------------------	---	--

Discussion

- A* implementation
 - One of the faster methods
 - Will find the shortest route from enemy to player location
 - Works on environments with various obstacles

Take Away

- How does this apply to us? Well our game is a top down 2D adventure RPG, mob AI and pathfinding is a big part of the experience and immersion of the game. We want mobs to feel and behave organically not just immediately track you down. Different enemy mobs are going to behave differently imparting a bit of their personality and features.
- Slimes will move lazily in all directions, almost at random until they see the player, from here they may move closer and pursue or lose interest allowing the player to pass by.
- Implementation:
 - Import the package from the official site
 - Prepare the tilemaps and the scene
 - Set up the pathfinder
 - Add components to the game object

Thank you

Algorithm	Typical Time Complexity	Notes
Breadth-First Search (BFS)	$O(N + E)$	Explores level by level; best for unweighted grids/graphs; guarantees shortest path in unweighted graphs.
Depth-First Search (DFS)	$O(N + E)$	Explores deep first; not optimal for shortest paths; can get “lost” in deep sections before finding target.
Dijkstra's Algorithm	$O((N + E) \log N)$ (with binary heap)	Always finds shortest path on weighted graphs; slower than informed heuristics; complexity depends on priority queue implementation.
A*	$O(b^d)$ worst case; in practice $\approx O(N + E)$	Heuristic-guided search; extremely efficient if heuristic is admissible and consistent ; complexity depends on heuristic quality (b = branching factor, d = depth of solution).
Best-First Search	$O(b^d)$ worst case	Greedy heuristic search (not guaranteed shortest); performance depends heavily on heuristic (can be faster than A* but less accurate).
D*	Similar to A* in many cases	Dynamic variant of A* for changing environments; reuses previous search results — efficiency depends on how often the environment changes.